

Syllabus (Spring 2025-2026)

CS 455 / CS 555

Large Language Models: Theoretical Foundations and Practical Applications

Instructor

İnanç Arın

e-mail: inancarin@sabanciuniv.edu

Course Description, Aim and Content

This course provides a comprehensive exploration of Large Language Models (LLMs), focusing on their foundational technologies, applications, and advanced techniques in deploying, fine-tuning, and evaluating them. Students will gain hands-on experience with transformer architectures, semantic search, retrieval-augmented generation (RAG), and AI agents. The course will also delve into cutting-edge methods for deploying LLMs locally, as well as leveraging Knowledge Graphs to enhance LLM-based systems for structured reasoning and improved retrieval.

By the end of the course, students will be able to design, implement, and evaluate complex LLM applications in real-world settings, from personalized chatbots to AI agents working in collaborative environments. The course will prepare students to tackle cutting-edge research and practical challenges in Natural Language Processing (NLP) and AI-driven systems.

Lecture Time & Place

- Wednesday 09:40-10:30 → UC G030
- Thursday 10:40-12:30 → FASS G062

Prerequisites

- **For CS 455**, students must have completed either CS 415 (Introduction to Deep Learning) or CS 412 (Machine Learning).
- **For CS 555**, students must have completed CS 512 (Machine Learning).

Course Content¹

1. Overview of Word Embeddings
 - a. Importance of semantic queries and analogies
 - b. How to compute word vectors
 - c. How to use pre-trained models
 - d. How to generate your own vector representations
 - e. Visualising word vectors
2. Introduction to Attention and Language Models
 - a. Mathematical Foundations of Transformers
 - i. Self-Attention, Multi-Head Attention, Positional Encodings
 - ii. Tokenization Strategies
3. Natural Language Understanding with BERT
 - a. The Masked Language Modeling Task
 - b. Next Sentence Prediction Task
 - c. Fine Tuning BERT for specific NLP tasks
 - i. BERT for sequence classification
 - ii. BERT for token classification
 - iii. BERT for QA
 - d. Flavors of BERT (DistilBERT, ALBERT, RoBERTa)
4. Natural Language Generation with GPT
 - a. The masked multi-head attention
 - b. Pre-training GPT
 - c. Few-shot learning
 - d. Fine-Tuning with Huggingface's Trainer
 - e. Sharing our models on HuggingFace
 - f. Case study: Classification problem like App Review Prediction

¹ The instructor may change the contents and the order of the topics during the semester.

- 
5. Semantic Search with LLMs
 - a. Retriever and Generator concepts
 - b. Introduction to Semantic Search
 - c. A Simple Indexer/Retriever
 6. Advanced Prompt Engineering and Implementing a RAG application
 - a. Chain-of-Thought Prompting
 - b. Optimizing Semantic Search with Cross-Encoders and Fine-Tuning
 - c. Designing a RAG ChatBot with open sourced and closed sourced LLMs
 - d. Case Study: Building a recommender system with open sourced and closed source embedding models
 - i. Fine-tuning an embedding model
 7. Foundations of LLM Evaluation
 - a. Evaluating Retrieval
 - b. Assessing Generative Responses
 - c. Evaluating Understanding Tasks (Embeddings, Classification etc)
 - d. Common Benchmarks and Evaluating LLMs with Benchmarks
 8. Running & Deploying LLMs Locally with Ollama & vLLM
 - a. Introduction to Local LLM Inference
 - i. Why run LLMs locally? (Privacy, Cost, Speed)
 - ii. Overview of Ollama and vLLM
 - b. Running models like Mistral, LLaMA with Ollama
 - c. High-Performance LLM Inference with vLLM
 - d. Quantization (4-bit, 8-bit models)
 - e. Case-study: Deploying a local chatbot using Ollama/vLLM
 9. Knowledge Graphs and LLMs
 - a. What are Knowledge Graphs? (Nodes, Edges, Relationships)
 - b. Difference between Graph Databases (Neo4j, ArangoDB) and traditional DBs
 - c. Why use Knowledge Graphs with LLMs? (Structured Reasoning, Explainability, Context Awareness)
 - d. Combining GraphDB + LLM to enhance retrieval
 - i. Developing RAG System with GraphDBs (GraphRAG)
 10. AI Agents
 - a. Introduction to Agents, Agent Architectures
 - b. Agent Frameworks (CrewAI, LangChain)
 - c. Agent Components: Thought, Action, Observation, Response
 - d. Handling multi-step reasoning with ReAct (Reasoning + Acting)
 - e. Evaluation of Agents

Grading²

Component	CS 455 Weight	CS 555 Weight
Homeworks	20%	15%
Midterm Exam	25%	20%
Final Exam	30%	25%
Term Project	25%	30%
Paper Presentation	-	10%

- Attendance will be taken during lectures. While it will not count toward your course grade, **you must attend at least 70% of the lectures to pass the course.**
- The **weighted exam average** will be calculated as
 - For CS 455 → $(0.25 * \text{Midterm_Exam} + 0.30 * \text{Final_Exam}) / 0.55$
 - For CS 555 → $(0.20 * \text{Midterm_Exam} + 0.25 * \text{Final_Exam}) / 0.45$
- If your **weighted exam average** is strictly below 40 (forty), you will automatically fail the course. However, achieving a **weighted exam average** score of 40 (forty) or above does not guarantee that you will pass the course.
- All of the assigned homework will be graded and taken into consideration in the overall grade. Each homework will have equal weight in overall grading.
- There won't be any make-up for attendance, homework, the project, and the paper presentation in any case, including medical health reports and official university activities.
- If you miss midterm or final examination, and if you do not take the make-up exam for that missing exam; then you will directly get an F grade.

² Please note that the grading schema is tentative and can be modified by the instructor as deemed necessary.

- There will be a make-up exam and this exam will be comprehensive (i.e. all topics included). You are allowed to miss only one exam, regardless of whether you have a medical report. Missing more than one exam will result in an automatic failure of the course.
 - In order to take the make-up exam, students must submit their medical reports to the university health center to validate them and notify the instructor within the day of the exam to be missed. The ones having other excuses should also contact the instructor within the day of the exam to be missed, and then the instructor will decide whether these students are allowed to take the make-up exam.
- The instructor has the right to have an oral interview for any grading item given in the syllabus. Students who will have the oral interview may be selected randomly or according to a suspicious situation observed by TAs or the instructors. After having the oral interview, the grade obtained by the student may change in a positive or a negative way.

Homework & Project Guidelines

1. AI Usage & Academic Integrity: "Glass Box" Policy

Generative AI tools (ChatGPT, Claude, Gemini, Copilot, etc.) are **permitted** and even encouraged (for homeworks and term projects) in this course as coding assistants. However, we adopt a "**Glass Box**" policy: usage must be transparent and documented.

- **Transparency:** You must submit a "**Prompt Log**" (explaining what you asked and how you modified the output) and a brief summary explaining which parts of the code were generated by AI and how you verified/modified them.
- **Oral Defense & Accountability:** The instructor reserves the right to conduct random **oral interviews** (viva voce) for any assignment or project.
 - During the interview, you will be asked to explain the logic of specific code blocks, hyperparameter choices, or architectural decisions.
 - **Crucial Rule:** If you cannot explain your own code, even if it runs perfectly, you will receive a grade of **zero** for that assignment. "The AI wrote it" is not a valid defense.

2. Homework Structure (Implementation + Analysis)

Homework assignments (individual assignments) in this course are designed to go beyond simple "code generation". Running the code successfully does not guarantee your grade. For every coding assignment, you must also submit an **Analysis Report** that includes:

- **Context-Specific Questions:** Each assignment sheet will contain specific questions that can only be answered by analyzing your unique execution logs (e.g., *"Compare the retrieval accuracy of your model for $k=3$ vs. $k=10$ on your specific dataset."*). Generic, AI-generated summaries will be penalized.
- **Reproducibility Evidence (Required):** Your Analysis Report must include a short "Run Evidence" section (e.g., command/config used, random seed, key hyperparameters, and an ipynb notebook with outputs or a screenshot). Submissions without run evidence may receive significant penalties.
- **Failure Analysis (The "Bad Cases" Table):** You are required to analyze where your model fails. You must provide a table of at least **5-10 failure cases** following this template:
 - *Input / Query*
 - *Model Output* (The wrong answer)
 - *Gold / Correct Answer*
 - *Error Type* (e.g., Hallucination, Retrieval Miss, Tokenization Issue)
 - *Root Cause Analysis if possible* (Why did it fail? e.g., "The context window was too short to include the answer")

For at least two failure cases, you must propose a fix and show before vs after evidence (qualitative examples and/or a quantitative change in a relevant metric).

3. Term Project Requirements

The term project is a long effort to build a significant LLM-based application. You should work in groups of 2 or 3 students.

➤ **Deliverables:**

- **Codebase:** Fully functional, reproducible code.
- **Prompt & Experiment Logs:** Documentation of your engineering process.
- **Final Report:** Must include the **Failure Analysis** described above and a quantitative evaluation against a baseline.
- **Live Demo:** A mandatory end-of-semester presentation.

➤ **Distinction between CS 455 and CS 555:**

- **CS 455 (Undergraduate):** Focus is on **Application & Engineering**. Students are expected to build a robust, end-to-end system (e.g., a specialized RAG chatbot, a semantic search engine for a specific domain) using existing frameworks and APIs.
- **CS 555 (Graduate):** Focus is on **Research & Innovation**. Merely wrapping an OpenAI API is **not sufficient**. Graduate projects must demonstrate technical depth, such as:
 - Fine-tuning open-source models (e.g., Llama, Mistral).
 - Developing custom Agentic workflows or novel RAG architectures.
 - Conducting rigorous benchmarks comparing multiple techniques.
- **For mixed groups** (if CS 455 & CS 555 students are working together), the project will be evaluated based on the stricter **CS 555 criteria** (requiring technical novelty).

Paper Presentation

1. **Assignment Description** Graduate (only CS 555) students will select a paper from a provided list of top-tier conferences (NeurIPS, ICLR, ACL), present its technical details to the class, and submit a 2-page critical review. The presentation should be approximately **15 minutes** long, covering the model architecture, results, and a critical analysis.
2. **Presentation Format & Delivery** The mode of presentation will be determined by the instructor:

- **In-Class Presentations:** A subset of papers, selected by the instructor, will be presented live during class hours in front of all students.
- **Offline/Video Presentations:** Students not selected for in-class sessions will either present outside of class hours or submit a recorded video presentation, based on the instructor's preference.
- **Review Report:** Regardless of the presentation mode, **all CS 555 students** must submit the 2-page critical review report.

3. Attendance & Final Exam Responsibility (ALL STUDENTS)

- **CS 455 Students:** Undergraduate students are **not** required to make a presentation or write a review.
- **Exam Coverage:** However, **attendance to in-class presentations is expected for ALL students (CS 455 & CS 555)**. Concepts, discussions, and technical details covered during these in-class presentations will be included in the **Final Exam**. Therefore, all students are responsible for understanding the core ideas of the presented papers.

AI Usage & Academic Integrity & Plagiarism Policy

As mentioned earlier, Generative AI tools (ChatGPT, Claude, Gemini, Copilot, etc.) **are allowed** in homeworks and term projects to assist with coding and debugging. However, the following rules apply strictly:

- **Documentation:** If you use AI to generate code, you must include a brief comment and a "Prompt Log" explaining what you asked and how you modified the output.
- **Oral Defense:** The instructor reserves the right to perform an **oral interview** for any assignment or project. During this interview, you must be able to explain the logic of every line of code. **Inability to explain your own submission (even if it works) will result in a grade of zero**, regardless of whether AI was used.
- **No "Copy-Paste" Tolerance:** Submitting code that you have not tested or do not understand is considered a violation of academic integrity.

Plagiarism means presenting someone else's work as yours. This is a very serious and ethical problem. When a plagiarism case is detected, sanctions are applied to all parties regardless of the actual source of the submission. These sanctions are as follows:

- For the midterm or final examination,
 - students directly fail the course, even in the first offense.
- For homework,
 - For the first time, all plagiarized submission owners receive a grade of 0 (zero),
 - For the second time, the student fails the course automatically.

Related Courses

- Stanford CME 295 - Transformers & Large Language Models ([CME 295](#))
- Stanford CS224N: Natural Language Processing with Deep Learning ([CS224n: Natural Language Processing with Deep Learning](#))
- Berkeley Large Language Model Agents ([Large Language Model Agents](#))
- HuggingFace LLM Course ([Introduction - Hugging Face LLM Course](#))
- HuggingFace AI Agents Course ([Welcome to the 😊 AI Agents Course](#))

Announcements and SUCourse+

Students are responsible for all announcements made during the regular class meetings. Students should check their Sabanci University e-mail accounts (mymail.sabanciuniv.edu) and follow the SUCourse+ site (<https://sucourse.sabanciuniv.edu/plus/>) for this class regularly as they are responsible for all announcements and postings delivered through these sources.

Important Note: This document may be modified during the semester (Spring 2025-2026) due to unforeseen reasons.